EMPLOYEE ATTRITION PREDICTION SYSTEM

Using MongoDB, PyMongo and Machine Learning

PROJECT REPORT

Name: Kashish Srivastava	**Student's Roll no:** 50
Reg. no: 12324693	**Academic Task No.:** 02
Course Code: CSE494	**Submitted To:** Dr. Ashu
Course Title: Intelligent NoSQL Databases	

1. Introduction

Employees are an important part of every organization. Their experience, skills, and contribution help a company achieve its goals. However, when employees leave, the organization may face increased recruitment costs, training expenses, and the loss of experienced workers. This situation is known as employee attrition.

Understanding patterns related to employee attrition can help organizations make better decisions. Data analysis and machine learning can examine employee information and identify patterns associated with attrition.

The Employee Attrition Prediction System is a data-driven project that uses MongoDB, Python, and machine learning to analyze employee records and predict whether an employee is likely to stay or leave. MongoDB stores records, PyMongo connects the database to Python, and machine learning models are trained and evaluated using the prepared data.

2. Project Objectives

1. Store employee records in MongoDB and manage them through PyMongo.
2. Retrieve employee data from MongoDB for analysis.
3. Check the dataset for missing values and duplicate records.
4. Prepare the data by removing unnecessary columns and encoding categorical values.
5. Train three machine learning models to predict employee attrition.
6. Evaluate the models using Accuracy, Precision, Recall, F1-score, and Mean Absolute Error (MAE).
7. Compare model performance using a table, graphs, and confusion matrices.
8. Identify a model that provides a useful balance between detecting employees who leave and making reliable predictions.

3. Dataset Description

The project uses the IBM HR Analytics Employee Attrition dataset, obtained from Kaggle. The dataset contains employee information, including work history, job roles, satisfaction levels, salary details, and other employment-related attributes. It consists of 1,470 records and 36 columns. Each record represents an individual employee, and the Attrition column indicates whether the employee left the organization or continued working there.

The dataset was downloaded from Kaggle and used for data preprocessing, analysis, and building machine learning models to predict employee attrition.

Dataset Source:
https://www.kaggle.com/datasets/pavansubhasht/ibm-hr-analytics-attrition-dataset

Property	Description
Dataset	HR Employee Attrition
Employee records	1,470
Original columns	36
Target column	Attrition
Target categories	Yes and No
Prediction type	Binary classification
Database	MongoDB
Database access	PyMongo

3.1 Important Dataset Attributes

- Personal and employment information: Age, Gender, Department, JobRole, and MaritalStatus.
- Salary and compensation information: MonthlyIncome, DailyRate, MonthlyRate, HourlyRate, and PercentSalaryHike.
- Work experience: TotalWorkingYears, YearsInCurrentRole, YearsWithCurrManager, and YearsSinceLastPromotion.
- Job satisfaction and working conditions: JobSatisfaction, EnvironmentSatisfaction, JobInvolvement, WorkLifeBalance, and OverTime.
- Target variable: Attrition, indicating whether an employee stayed or left.

3.2 Target Variable

The target variable is Attrition. 'No' means the employee stayed, while 'Yes' means the employee left. For modeling, these labels were mapped to 0 and 1 respectively.

4. Tools and Technologies Used

Technology	Purpose
MongoDB	Stores employee records in a document-oriented database.
PyMongo	Connects Python to MongoDB and performs database operations.
Python	Used for data processing, model training, and evaluation.

Jupyter Notebook in VS Code	Provides an environment to execute code step by step.
Pandas	Reads, organizes, and analyzes employee data.
Scikit-learn	Provides machine learning models and evaluation metrics.
Matplotlib	Creates charts for comparing model performance.
Seaborn	Creates heatmaps for confusion matrices.

5. MongoDB and PyMongo Implementation

MongoDB was used as the database component. It stores records as documents, making it convenient to work with employee information. PyMongo connects the Python notebook to the local MongoDB server, allowing records to be inserted, retrieved, counted, and summarized.

5.1 Connecting to MongoDB

A MongoClient connection was created for the local MongoDB server. A ping command was executed to verify that the server was reachable before running further database operations.



Explanation: The output confirms that Python successfully connected to the local MongoDB server.

5.2 Creating the Database and Collection

The database EmployeeAttritionDB and collection employees were selected. The collection stores each employee as a separate document containing fields such as Age, Department, JobRole, MonthlyIncome, and Attrition.

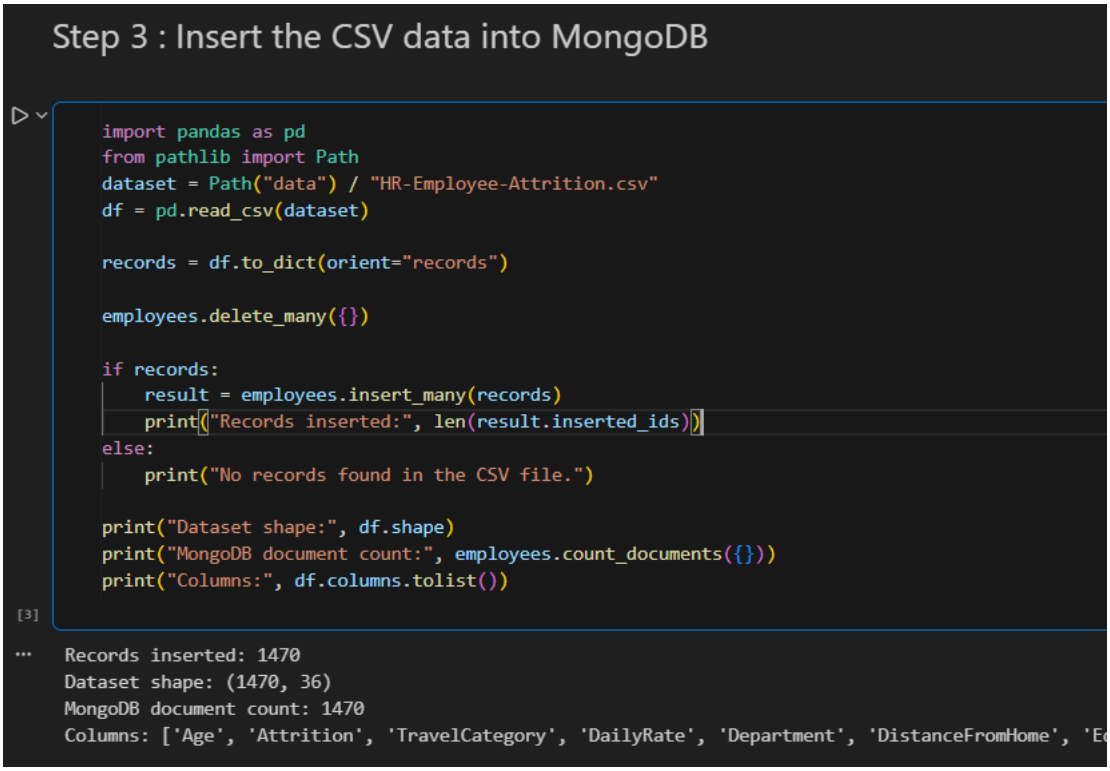


Explanation: This step prepares the database and collection for storing the employee records.

5.3 Importing the Dataset into MongoDB

The CSV file was loaded with Pandas and converted into a list of dictionaries using to_dict(orient='records'). The records were then inserted into MongoDB using insert_many(). The collection was cleared before re-importing to prevent duplicate copies when the import cell was rerun.

The import output confirmed that 1,470 employee records were inserted.



Explanation: This step transfers employee data from the CSV file into MongoDB for database queries and analysis.

5.4 Retrieving and Analyzing Records

PyMongo was used to count documents, retrieve a sample employee record, and summarize attrition using an aggregation pipeline with the $group stage.

Attrition status	Number of employees
Stayed (No)	1,233
Left (Yes)	237
Total	1,470

The dataset contains more employees who stayed than employees who left. This class imbalance is important when evaluating models because predicting the majority class alone would not be sufficient.

Step 4 : Check the dataset using PyMongo

```
total_employees = employees.count_documents({})
print("Total employees:", total_employees)

# Display one employee document
print("Sample employee record:")
print(employees.find_one({}, {"_id": 0}))

print("Attrition distribution:")
pipeline = [
    {
        "$group": {
            "_id": "$Attrition",
            "count": {"$sum": 1}
        }
    }
]
for record in employees.aggregate(pipeline):
    print(record["_id"], ":", record["count"])
```

[4] ✓ 0.0s

```
Total employees: 1470
Sample employee record:
{'Age': 41, 'Attrition': 'Yes', 'TravelCategory': 'Travel_Rarely', 'DailyRate': 1102, 'Depart
Attrition distribution:
No : 1233
Yes : 237
```

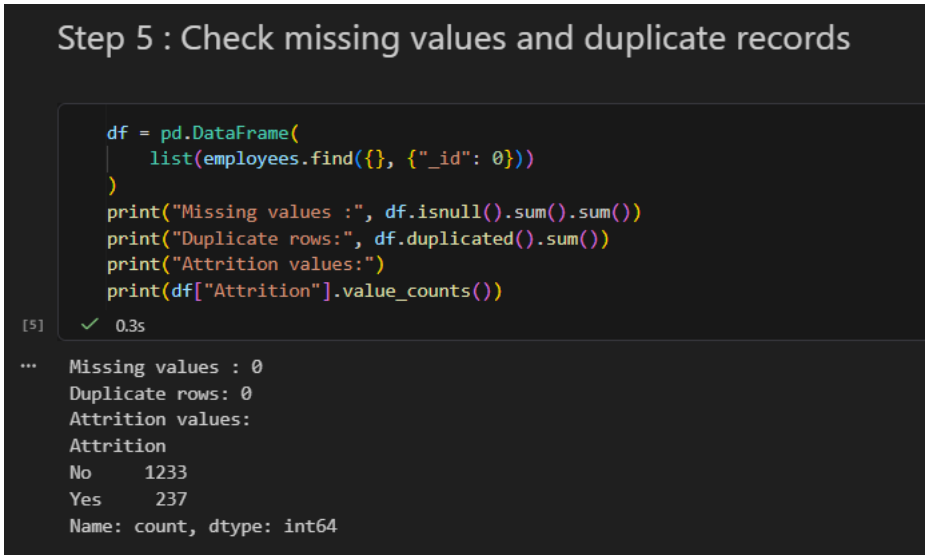
Explanation: This demonstrates how MongoDB can retrieve employee records and summarize attrition directly from the collection.

6. Data Preprocessing

Before training the models, the employee data was checked and prepared. Preprocessing reduces unnecessary information and converts the data into a format suitable for machine learning.

6.1 Missing Values and Duplicate Checks

The records were loaded into a Pandas DataFrame and checked for missing values and duplicate rows. The checks found zero missing values and zero duplicate rows, so no imputation or duplicate removal was required.



Explanation: These checks confirm that the dataset did not require additional treatment for missing or duplicate records.

6.2 Removing Unnecessary Columns

EmployeeNumber was removed because it is an identifier rather than a meaningful predictor. EmployeeCount, Over18, and StandardHours were also removed because they contained constant values. After this step, the working dataset contained 32 columns, including the target. The input features had 31 columns before encoding.

Step 6 : Prepare the features and target

```
data = df.copy()
# Remove the employee identifier because it is not a useful predictor
data.drop(columns=["EmployeeNumber"], inplace=True, errors="ignore")

# Remove constant columns that provide no predictive information
constant_cols = [
    col for col in data.columns
    if data[col].nunique(dropna=False) <= 1
]
data.drop(columns=constant_cols, inplace=True)

print("Constant columns removed:", constant_cols)
print("Remaining columns:", len(data.columns))
print("Remaining shape:", data.shape)

X = data.drop(columns=["Attrition"])
y = data["Attrition"].map({"No": 0, "Yes": 1})

print("\nFeature shape:", X.shape)
print("Target shape:", y.shape)
print("\nTarget values:")
print(y.value_counts(dropna=False))
```

[6] ✓ 0.0s

```
Constant columns removed: ['EmployeeCount', 'Over18', 'StandardHours']
Remaining columns: 32
Remaining shape: (1470, 32)

Feature shape: (1470, 31)
Target shape: (1470,)

Target values:
Attrition
0    1233
1     237
Name: count, dtype: int64
```

Explanation: Removing these columns keeps the model focused on features that may help distinguish attrition outcomes.

6.3 Separating Features and Target

The input features were stored in X and the target labels in y. The target values were mapped from No to 0 and Yes to 1.

Step 7 : Identify numerical and categorical columns

```
numeric_cols = X.select_dtypes(include=["number"]).columns.tolist()
categorical_cols = X.select_dtypes(
    include=["object", "category", "bool"]
).columns.tolist()

print("Numerical columns:", len(numeric_cols))
print(numeric_cols)

print("\nCategorical columns:", len(categorical_cols))
print(categorical_cols)

print("\nMissing values in X:", X.isnull().sum().sum())
print("Missing values in y:", y.isnull().sum())
```

[7] ✓ 0.0s

```
Numerical columns: 23
['Age', 'DailyRate', 'DistanceFromHome', 'Education', 'EnvironmentSatisfaction', 'HourlyRate', 'JobInvolvement', 'JobLevel', '

Categorical columns: 8
['TravelCategory', 'Department', 'EducationField', 'Gender', 'JobRole', 'MaritalStatus', 'OverTime', 'ExperienceGroup']

Missing values in X: 0
Missing values in y: 0
```

Explanation: This separates the information used to make predictions from the outcomes the models need to learn.

6.4 Encoding Categorical Variables

The categorical features were converted to numerical indicator columns using pandas.get_dummies() with drop_first=True. This produced 46 encoded input features from the 31 original input columns.

Step 8 : Preprocessing

```
# Convert categorical columns into numerical columns
X_encoded = pd.get_dummies(X, drop_first=True, dtype=int)

print("Original features:", X.shape[1])
print("Features after encoding:", X_encoded.shape[1])
print("\nPreprocessing completed!")
print("Encoded dataset shape:", X_encoded.shape)
display(X_encoded.head(10))
```

✓ 0.1s

```
Original features: 31
Features after encoding: 46

Preprocessing completed!
Encoded dataset shape: (1470, 46)
```

	Age	DailyRate	DistanceFromHome	Education	EnvironmentSatisfaction	HourlyRate	JobInvolvement	JobLevel	JobSatisfaction	MonthlyIncome	...	JobRole_Manufacturing Director
0	41	1102	1	2	2	94	3	2	4	5993	...	0
1	49	279	8	1	3	61	2	2	2	5130	...	0
2	37	1373	2	2	4	92	2	1	3	2090	...	0
3	33	1392	3	4	4	56	3	1	3	2909	...	0
4	27	591	2	1	1	40	3	1	2	3468	...	0
5	32	1005	2	2	4	79	3	1	4	3068	...	0
6	59	1324	3	3	3	81	4	1	1	2670	...	0
7	30	1358	24	1	4	67	3	1	3	2693	...	0
8	38	216	23	3	4	44	2	3	3	9526	...	1
9	36	1299	27	3	3	94	3	2	3	5237	...	0

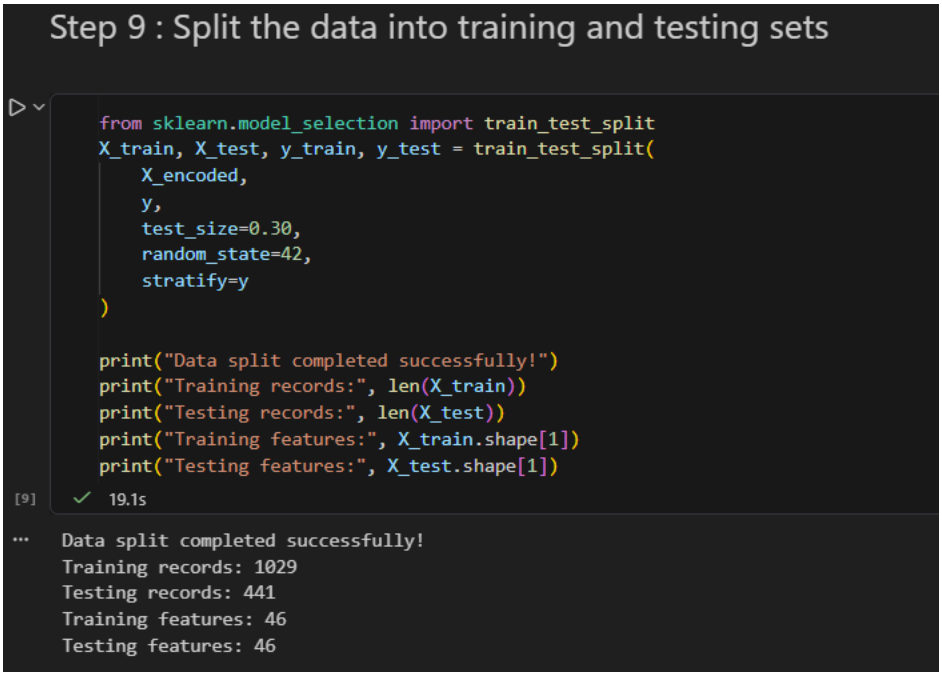
10 rows × 46 columns

Explanation: Encoding makes text-based categories usable by the classification algorithms.

6.5 Train-Test Split

The data was divided into 70% training data and 30% testing data. Stratification was used to preserve the class distribution, and random_state=42 made the split reproducible.

Dataset portion	Records
Training set (70%)	1,029
Testing set (30%)	441
Total	1,470



Explanation: The models learn from the training set and are evaluated on the separate test set.

7. Machine Learning Model Implementation

Three classification models were implemented and evaluated using the same training and testing split. Their results were compared to understand the different trade-offs in predicting employee attrition.

7.1 Logistic Regression

Logistic Regression was used as the baseline model. It estimates the probability of an employee belonging to a class and predicts whether the employee will stay or leave. A pipeline combined StandardScaler with LogisticRegression, and max_iter was set to 3000.

Step 10 : Train the Logistic Regression model

```
from sklearn.pipeline import make_pipeline
from sklearn.preprocessing import StandardScaler
from sklearn.linear_model import LogisticRegression

# Create the Logistic Regression model
model = make_pipeline(
    StandardScaler(),
    LogisticRegression(
        max_iter=3000,
        random_state=42
    )
)
model.fit(X_train, y_train)
print("Logistic Regression model trained successfully.")
```

[10] ✓ 0.7s

```
Logistic Regression model trained successfully.
```

Step 11 : Generate Prediction

```
y_pred = model.predict(X_test)

print("Actual values:", y_test.values[:10])
print("Predicted values:", y_pred[:10])
print("Total predictions:", len(y_pred))
```

[11] ✓ 0.0s

```
Actual values: [0 0 0 0 0 0 0 0 0 0]
Predicted values: [0 0 0 0 0 0 0 0 0 0]
Total predictions: 441
```

Explanation: This model establishes a baseline for comparing the other approaches.

7.2 Decision Tree

The Decision Tree classifier predicts outcomes by splitting records according to feature conditions. The model used max_depth=6, min_samples_leaf=5, and class_weight='balanced'. These settings limit tree complexity and account for the imbalanced target classes.

Step 13 : Train a Decision Tree model

```
from sklearn.tree import DecisionTreeClassifier
dt_model = DecisionTreeClassifier(
    max_depth=6,
    min_samples_leaf=5,
    class_weight="balanced",
    random_state=42
)
dt_model.fit(X_train, y_train)
dt_pred = dt_model.predict(X_test)
print("Decision Tree model trained successfully.")
print("Total predictions:", len(dt_pred))
```

[13] ✓ 1.6s

```
Decision Tree model trained successfully.
Total predictions: 441
```

Explanation: The Decision Tree offers a different classification approach for comparison with Logistic Regression.

7.3 Improved Logistic Regression

The second model used class_weight='balanced'. Since the dataset has fewer employees who left than employees who stayed, this setting gives greater consideration to the minority class during training. This can improve recall for employees who left, although accuracy or precision may decrease.

Step 16 : Improved Logistic Regression

```
improved_model = make_pipeline(
    StandardScaler(),
    LogisticRegression(
        max_iter=3000,
        class_weight="balanced",
        random_state=42
    )
)
improved_model.fit(X_train, y_train)
improved_pred = improved_model.predict(X_test)
print("Improved Logistic Regression model trained successfully.")
```

```
[18]  ✓ 0.1s
...  Improved Logistic Regression model trained successfully.
```

Explanation: This version aims to identify more employees who left rather than optimizing accuracy alone.

8. Evaluation Results

8.1 Evaluation Metrics

- Accuracy: The proportion of all predictions that were correct.
- Precision: Among employees predicted to leave, the proportion who actually left.
- Recall: Among employees who actually left, the proportion correctly identified by the model.
- F1-score: The harmonic mean of Precision and Recall, useful when classes are imbalanced.
- Mean Absolute Error (MAE): The average absolute difference between actual binary labels and predicted labels; lower is better.

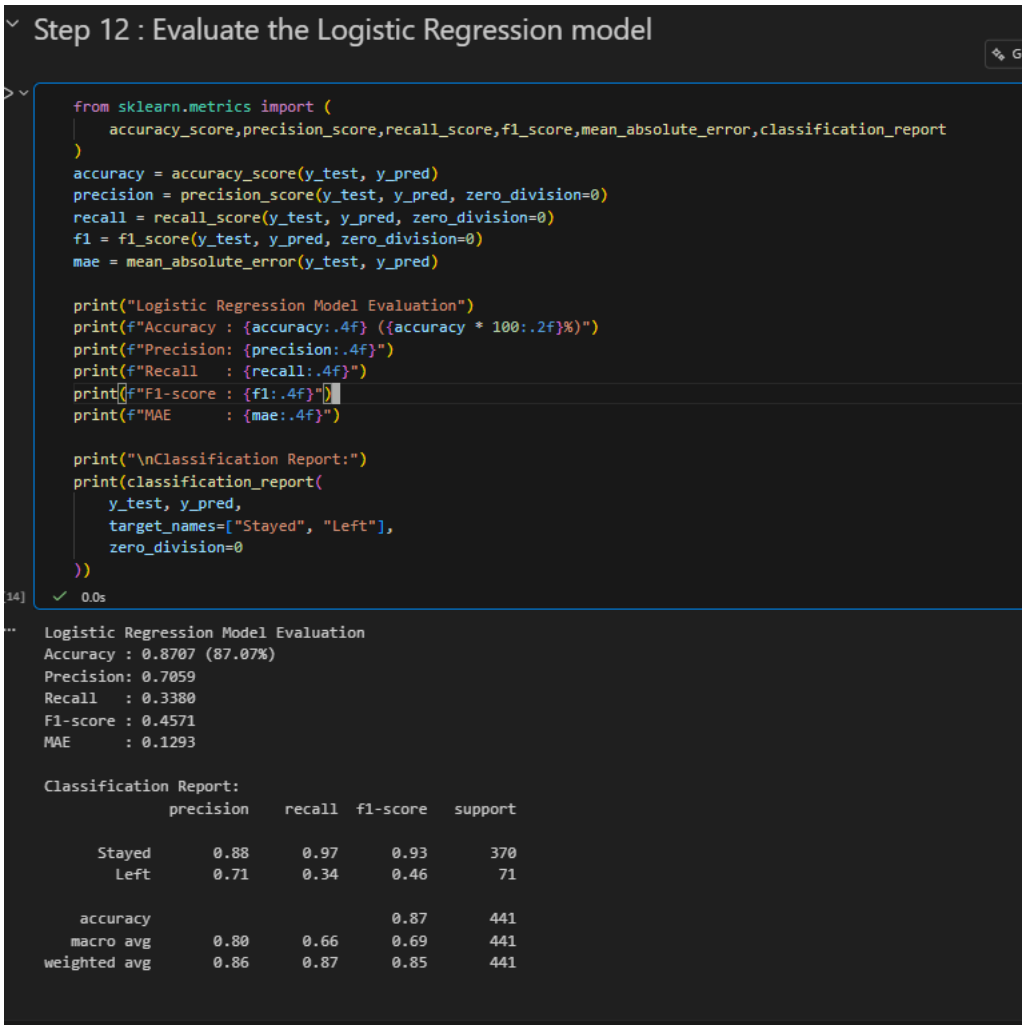
8.2 Model Results

All models were evaluated on the same test set of 441 employees. The results below are from the project notebook.

Model	Accuracy	Precision	Recall	F1-score	MAE
Logistic Regression	0.8707	0.7059	0.3380	0.4571	0.1293
Improved Logistic Regression	0.7800	0.3917	0.6620	0.4921	0.2200
Decision Tree	0.7302	0.3125	0.5634	0.4020	0.2698

8.3 Logistic Regression

The baseline Logistic Regression achieved 87.07% accuracy, precision of 0.7059, recall of 0.3380, and an F1-score of 0.4571. It performed well on the majority class, but it identified only a portion of employees who actually left.



Explanation: The classification report provides a closer look at performance for both attrition classes.

8.4 Decision Tree

The Decision Tree achieved 73.02% accuracy, precision of 0.3125, recall of 0.5634, and an F1-score of 0.4020. Its recall was higher than the baseline model's, but its overall accuracy, precision, and F1-score were lower.

Step 14 : Evaluate the Decision Tree model

```
from sklearn.metrics import (
    accuracy_score,
    precision_score,
    recall_score,
    f1_score,
    mean_absolute_error,
    classification_report
)
dt_accuracy = accuracy_score(y_test, dt_pred)
dt_precision = precision_score(y_test, dt_pred, zero_division=0)
dt_recall = recall_score(y_test, dt_pred, zero_division=0)
dt_f1 = f1_score(y_test, dt_pred, zero_division=0)
dt_mae = mean_absolute_error(y_test, dt_pred)

print("Decision Tree model evaluation.")
print(f"Accuracy : {dt_accuracy:.4f} ({dt_accuracy * 100:.2f}%)")
print(f"Precision: {dt_precision:.4f}")
print(f"Recall   : {dt_recall:.4f}")
print(f"F1-score : {dt_f1:.4f}")
print(f"MAE      : {dt_mae:.4f}")

print("\nClassification Report:")
print(classification_report(
    y_test,
    dt_pred,
    target_names=["Stayed", "Left"],
    zero_division=0
))
```

[16] ✓ 0.0s

```
Decision Tree model evaluation.
Accuracy : 0.7302 (73.02%)
Precision: 0.3125
Recall   : 0.5634
F1-score : 0.4020
MAE      : 0.2698

Classification Report:
              precision    recall  f1-score   support

      Stayed       0.90      0.76      0.83       370
        Left       0.31      0.56      0.40        71

    accuracy                           0.73       441
   macro avg       0.61      0.66      0.61       441
weighted avg       0.81      0.73      0.76       441
```

Explanation: This output shows the Decision Tree's ability to identify employees who leave and its overall prediction trade-offs.

8.5 Improved Logistic Regression

Improved Logistic Regression achieved 78.00% accuracy, precision of 0.3917, recall of 0.6620, and an F1-score of 0.4921. Its recall and F1-score were the highest among the three models, indicating that it identified a larger share of employees who left.

Step 17 : Evaluate the Improved Logistic Regression

```
from sklearn.metrics import (
    accuracy_score,
    precision_score,
    recall_score,
    f1_score,
    mean_absolute_error
)
improved_accuracy = accuracy_score(y_test, improved_pred)
improved_precision = precision_score(
    y_test, improved_pred, zero_division=0
)
improved_recall = recall_score(
    y_test, improved_pred, zero_division=0
)
improved_f1 = f1_score(
    y_test, improved_pred, zero_division=0
)
improved_mae = mean_absolute_error(y_test, improved_pred)

print("Improved Logistic Regression model evaluation.")
print(f"Accuracy : {improved_accuracy:.4f}")
print(f"Precision: {improved_precision:.4f}")
print(f"Recall   : {improved_recall:.4f}")
print(f"F1-score : {improved_f1:.4f}")
print(f"MAE      : {improved_mae:.4f}")
```

[20] ✓ 0.0s

```
Improved Logistic Regression model evaluation.
Accuracy : 0.7800
Precision: 0.3917
Recall   : 0.6620
F1-score : 0.4921
MAE      : 0.2200
```

Explanation: The results show the trade-off created by class balancing: more employees who left are detected, but more false positives may occur.

8.6 Confusion Matrix Comparison

A confusion matrix compares actual labels with predicted labels. True negatives are employees who stayed and were predicted to stay; false positives are employees who stayed but were predicted to leave; false negatives are employees who left but were predicted to stay; and true positives are employees who left and were predicted to leave.

The baseline Logistic Regression confusion matrix was:

Actual / Predicted	**Stayed (0)**	**Left (1)**
Stayed (0)	360	10
Left (1)	47	24

The baseline model correctly identified 360 employees who stayed and 24 who left. It missed 47 employees who actually left and incorrectly flagged 10 employees who stayed. The combined confusion matrix figure compares these types of outcomes across all three models.

Step 19 : Confusion Matrix of all three models

```
import matplotlib.pyplot as plt
import seaborn as sns
from sklearn.metrics import confusion_matrix

models = {
    "Logistic Regression": y_pred,
    "Improved Logistic Regression": improved_pred,
    "Decision Tree": dt_pred
}

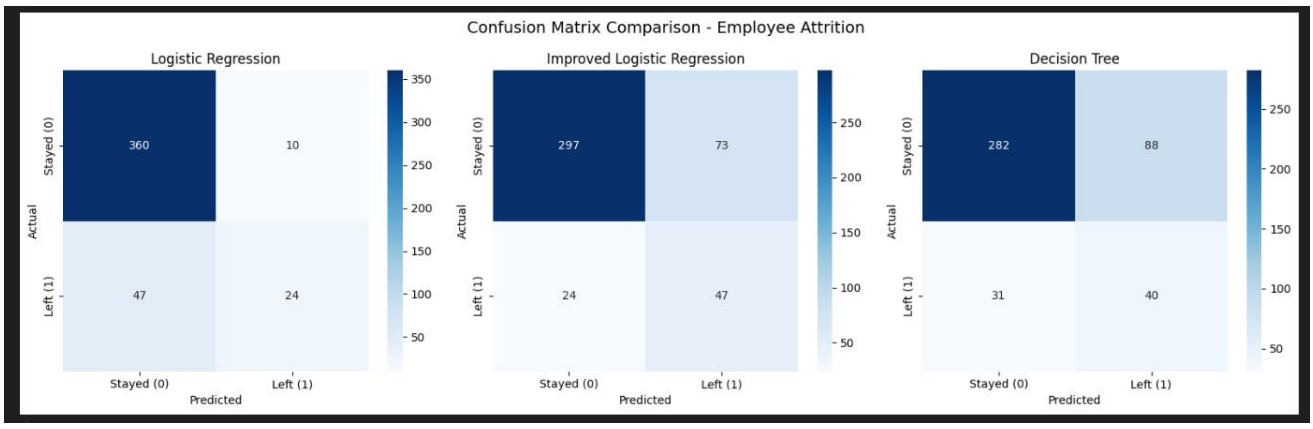
fig, axes = plt.subplots(1, 3, figsize=(16, 5))

for ax, (model_name, predictions) in zip(axes, models.items()):
    cm = confusion_matrix(y_test, predictions, labels=[0, 1])

    sns.heatmap(
        cm,
        annot=True,
        fmt="d",
        cmap="Blues",
        xticklabels=["Stayed (0)", "Left (1)"],
        yticklabels=["Stayed (0)", "Left (1)"],
        ax=ax
    )

    ax.set_title(model_name)
    ax.set_xlabel("Predicted")
    ax.set_ylabel("Actual")

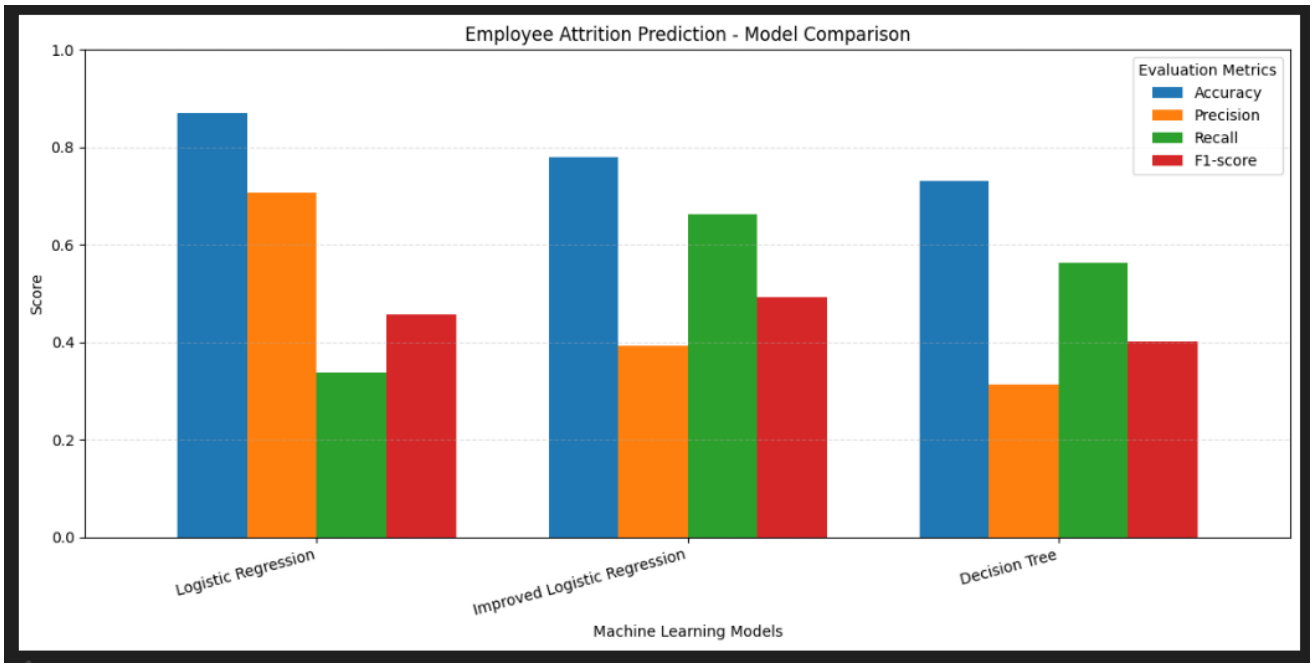
plt.suptitle("Confusion Matrix Comparison - Employee Attrition", fontsize=14)
plt.tight_layout()
plt.show()
```



Explanation: The side-by-side heatmaps show how often each model correctly identifies or misclassifies employees in each class.

8.7 Model Comparison Graph

A grouped bar chart compares Accuracy, Precision, Recall, and F1-score across the three models. It visually highlights the higher recall and F1-score of Improved Logistic Regression and the higher accuracy of the baseline model.



Explanation: The chart provides a quick visual summary of the differences between the models.

9. Results and Discussion

The project successfully connected MongoDB to Python, imported the employee records, performed database queries, prepared the dataset, and evaluated three classification models. The dataset contained 1,470 records, including 237 employees who left and 1,233 who stayed.

The baseline Logistic Regression model had the highest accuracy and precision. However, Improved Logistic Regression had the highest recall and F1-score, meaning it identified more of the employees who actually left. For a project focused on detecting potential attrition, Improved Logistic Regression is a reasonable choice among the tested models, although its lower precision means that some employees predicted to leave stayed.

The results describe patterns in this dataset and should not be treated as proof of why a particular employee leaves. The predictions should be used as signals for further analysis, not as the sole basis for employment decisions.

10. Conclusion

The Employee Attrition Prediction System demonstrates how MongoDB, PyMongo, Python, and machine learning can be combined to analyze employee data. MongoDB stored and retrieved the employee records, PyMongo connected the database to the notebook, and preprocessing prepared the data for modeling.

Three models—Logistic Regression, Improved Logistic Regression, and Decision Tree—were trained and compared. The baseline Logistic Regression achieved the highest accuracy of 87.07%, while Improved Logistic Regression achieved the highest recall of 66.20% and F1-score of 0.4921. This shows that the best model depends on the project's goal: overall accuracy or stronger identification of employees who leave.

Overall, the project provided practical experience in database management, data preprocessing, classification, and model evaluation. It also demonstrated the importance of considering multiple evaluation metrics when working with imbalanced data.